Experiment 1 ? Program 8

DATA SCIENCE - 22DS102006

Web Scraping using Beautiful Soup

Extract product reviews from Amazon website and save to a file.

Perform Exploratory Data Analysis on extracted product reviews.

Generate WordCloud for all reviews, positive and negative reviews.

Plot the distribution of stopwords, numerics, wordcount values, charactercount values and average wordlength.

Display the sentiment value using nltk and vader.

IV. Create Scatter Intensity Plot of Sentiments.

Web Scraping Using Beautiful Soup and Exploratory Data Analysis of Product Reviews

This practical record contains the experiments for extracting product reviews, performing text analysis, visualizing positive and negative reviews, and analyzing sentiment using NLTK and VADER.

Important note: Amazon may restrict automated scraping through its terms, robots.txt, and anti-bot protections. The first experiment uses a locally saved HTML file or an authorized review data source rather than bypassing access controls. The remaining experiments work with the extracted review dataset saved as a CSV file.

Software requirements

Python 3.x

Jupyter Notebook / Google Colab

Libraries: beautifulsoup4, pandas, requests, matplotlib, seaborn, wordcloud, nltk, vaderSentiment, numpy

Install the required libraries:

```
pip install beautifulsoup4 pandas requests matplotlib seaborn wordcloud nltk vaderSentiment numpy
```

Experiment 1: Web Scraping Using Beautiful Soup

Aim

To extract product reviews and related information from an authorized HTML page or permitted review source using Beautiful Soup and save the extracted data into a CSV file for further analysis.

Theory

Beautiful Soup is a Python library used to parse HTML and XML documents. It allows users to locate HTML elements using tags, classes, IDs, and CSS selectors.

The web scraping process involves:

Obtaining HTML from a permitted source.

Parsing the HTML using Beautiful Soup.

Identifying the product review elements.

Extracting the review text, rating, and other available fields.

Saving the collected data to a CSV file.

Syntax

```
from bs4 import BeautifulSoupsoup = BeautifulSoup(html_content, "html.parser")elements =
soup.select("CSS_SELECTOR")text = elements[0].get_text(strip=True)
```

Here, html_content contains the page HTML, while CSS_SELECTOR identifies the elements to extract.

Source code

For a reproducible experiment, save an authorized review page as reviews.html and run the following code. The CSS selectors must match the structure of the saved page.

from bs4 import BeautifulSoup

```
import pandas as pd

# Read the permitted, locally saved HTML file

with open("reviews.html", "r", encoding="utf-8") as file:

html_content = file.read()

soup = BeautifulSoup(html_content, "html.parser")

# Update these selectors to match the source HTML

review_blocks = soup.select(".review")

records = []

for block in review_blocks:

review_element = block.select_one(".review-text")

rating_element = block.select_one(".rating")

title_element = block.select_one(".review-title")

review_text = (

review_element.get_text(" ", strip=True)

if review_element else ""

)

rating_text = (

rating_element.get_text(" ", strip=True)

if rating_element else ""
```

```
)
title = (
title_element.get_text(" ", strip=True)
if title_element else ""
)
if review_text:
records.append({
"title": title,
"review": review_text,
"rating": rating_text
})
df = pd.DataFrame(records)
df.to_csv("product_reviews.csv", index=False, encoding="utf-8-sig")
print(df.head())
print("Total reviews extracted:", len(df))
print("Saved to product_reviews.csv")
```

Expected output

	title	review	rating
0	Great product	Excellent quality...	5 out of 5
1	Not satisfied	The product stopped...	2 out of 5

```
2         Worth buying    Good performance...       4 out of 5
```

```
Total reviews extracted: 3
```

```
Saved to product_reviews.csv
```

The displayed records are illustrative. Actual output depends on the HTML file and its CSS selectors.

Explanation

BeautifulSoup() parses the HTML document.

select(".review") identifies review containers.

select_one() retrieves individual fields within each review.

get_text() extracts readable text while removing HTML markup.

DataFrame() organizes the extracted information into a table.

to_csv() saves the dataset for subsequent experiments.

Result

Product review data was extracted from a permitted HTML source using Beautiful Soup and saved in product_reviews.csv for further exploratory data analysis.

Experiment 2: Exploratory Data Analysis and WordCloud Generation

Aim

To perform exploratory data analysis (EDA) on product reviews and generate WordCloud visualizations for all reviews, positive reviews, and negative reviews.

Theory

Exploratory Data Analysis helps identify patterns, frequent terms, and differences between positive and negative customer feedback. A WordCloud represents words visually, with frequently occurring words displayed more prominently.

For sentiment grouping, this experiment uses VADER compound scores:

Positive: compound score greater than or equal to 0.05.

Negative: compound score less than or equal to −0.05.

Neutral: scores between −0.05 and 0.05.

Syntax

from wordcloud import WordCloud

cloud = WordCloud(

width=1000,

height=500,

background_color="white"

).generate(text)

cloud.to_file("wordcloud.png")

Source code

Run this code after completing Experiment 1. It creates three WordClouds and saves the sentiment labels for reuse in later experiments.

```
import pandas as pd

import matplotlib.pyplot as plt

from wordcloud import WordCloud

from vaderSentiment.vaderSentiment import
```

```
SentimentIntensityAnalyzer

# Load extracted reviews

df = pd.read_csv("product_reviews.csv")

df["review"] = df["review"].fillna("").astype(str)

# Initialize sentiment analyzer

analyzer = SentimentIntensityAnalyzer()

def classify_sentiment(text):

score = analyzer.polarity_scores(text)["compound"]

if score >= 0.05:

return "Positive"

elif score <= -0.05:

return "Negative"

return "Neutral"

df["sentiment"] = df["review"].apply(classify_sentiment)

def generate_wordcloud(text, title, filename):

if not text.strip():

print(f"No text available for: {title}")

return

cloud = WordCloud(
```

```
    width=1200,
    height=600,
    background_color="white",
    max_words=150,
    collocations=False
    ).generate(text)
    plt.figure(figsize=(12, 6))
    plt.imshow(cloud, interpolation="bilinear")
    plt.axis("off")
    plt.title(title)
    plt.tight_layout()
    plt.savefig(filename, dpi=300, bbox_inches="tight")
    plt.show()
    # All reviews
    all_text = " ".join(df["review"])
    generate_wordcloud(
    all_text, "WordCloud - All Reviews", "wordcloud_all.png"
    )
    # Positive reviews
```

```
positive_text = " ".join(
df.loc[df["sentiment"] == "Positive", "review"]
)
generate_wordcloud(
positive_text, "WordCloud - Positive Reviews",
"wordcloud_positive.png"
)
# Negative reviews
negative_text = " ".join(
df.loc[df["sentiment"] == "Negative", "review"]
)
generate_wordcloud(
negative_text, "WordCloud - Negative Reviews",
"wordcloud_negative.png"
)
df.to_csv("product_reviews_analyzed.csv", index=False)
print(df["sentiment"].value_counts())
print("WordCloud images saved successfully.")
```

Expected output

Three visualizations will be generated:



1. WordCloud — All Reviews

Displays frequently occurring terms across the entire dataset, such as product, quality, good, price, and delivery, depending on the actual reviews.



2. WordCloud — Positive Reviews

Highlights favorable expressions such as excellent, great, useful, and satisfied, when present.



3. WordCloud — Negative Reviews

Highlights unfavorable expressions such as poor, bad, broken, and disappointed, when present.

These images illustrate the expected style only. Running the Python code produces the actual WordClouds from your dataset.

Explanation

SentimentIntensityAnalyzer() calculates sentiment polarity using VADER.

The classify_sentiment() function assigns a positive, negative, or neutral label.

generate_wordcloud() converts review text into a frequency-based visualization.

collocations=False reduces the appearance of repeated word pairs.

savefig() exports each visualization at 300 DPI.

The analyzed dataset is saved as product_reviews_analyzed.csv.

Result

Exploratory text analysis was performed, and WordCloud visualizations for all, positive, and negative product reviews were generated and saved.

Experiment 3: Distribution of Stopwords, Numerics, Word Count, Character Count, and Average Word Length

Aim

To analyze product reviews by calculating stopword frequency, numeric-token frequency, word count, character count, and average word length, and to visualize their distributions.

Theory

Text statistics provide useful information about review structure and writing patterns.

Stopwords: Common words such as the, is, and, and of.

Numerics: Numeric tokens such as 10, 2026, or 100.

Word count: Total number of words in a review.

Character count: Number of characters in a review, including spaces.

Average word length: Mean number of characters per word.

These measurements can reveal whether reviews are brief or detailed and whether numerical expressions occur frequently.

Syntax

from nltk.corpus import stopwords

stop_words = set(stopwords.words("english"))

word_count = len(text.split())

character_count = len(text)

average_word_length = word_count and character_count / word_count

For accurate average word length, the implementation below calculates the mean length of alphabetic or alphanumeric word tokens rather than dividing total characters by word count.

Source code

```
import re

import pandas as pd

import nltk

import matplotlib.pyplot as plt

import seaborn as sns

from nltk.corpus import stopwords

from collections import Counter

nltk.download("stopwords", quiet=True)

df = pd.read_csv("product_reviews.csv")

df["review"] = df["review"].fillna("").astype(str)

stop_words = set(stopwords.words("english"))

def analyze_review(text):

tokens = re.findall(r"\b[\w']+\b", text.lower())

words = re.findall(r"\b[A-Za-z]+\b", text)

numeric_tokens = re.findall(r"\b\d+(?:\.\d+)?\b", text)

stopword_tokens = [

token for token in tokens if token in stop_words

]
```

```
return pd.Series({
"word_count": len(tokens),
"character_count": len(text),
"numeric_count": len(numeric_tokens),
"stopword_count": len(stopword_tokens),
"average_word_length": (
sum(len(word) for word in words) / len(words)
if words else 0
)
})
statistics = df["review"].apply(analyze_review)
df = pd.concat([df, statistics], axis=1)
# Save calculated statistics
df.to_csv("review_statistics.csv", index=False)
# Plot distributions
columns = [
("stopword_count", "Stopword Count"),
("numeric_count", "Numeric Token Count"),
("word_count", "Word Count"),
```

```
("character_count", "Character Count"),
("average_word_length", "Average Word Length")
]
fig, axes = plt.subplots(3, 2, figsize=(14, 15))
axes = axes.flatten()
for i, (column, title) in enumerate(columns):
sns.histplot(
data=df, x=column, kde=True,
ax=axes[i], bins=20
)
axes[i].set_title(title)
axes[i].set_xlabel(title)
axes[i].set_ylabel("Number of Reviews")
axes[-1].axis("off")
plt.tight_layout()
plt.savefig("review_statistics_distributions.png", dpi=300)
plt.show()
# Most common stopwords
all_tokens = re.findall(
```

```
r"\b[\w']+\b",

" ".join(df["review"]).lower()

)

stopword_frequency = Counter(

token for token in all_tokens if token in stop_words

)

print("Top 10 stopwords:")

print(stopword_frequency.most_common(10))

print("\nDescriptive statistics:")

print(df[[

"stopword_count",

"numeric_count",

"word_count",

"character_count",

"average_word_length"

]].describe())
```

Expected output

The program produces a figure containing five histograms:

Stopword-count distribution.

Numeric-token-count distribution.

Word-count distribution.

Character-count distribution.

Average-word-length distribution.

It also prints the ten most common stopwords and a statistical summary containing the count, mean, standard deviation, minimum, quartiles, and maximum.

Example of the output format:

```
Top 10 stopwords:
```

```
[('the', 120), ('and', 85), ('is', 73), ...]
```

```
Descriptive statistics:
```

```
       stopword_count  numeric_count  word_count  ...
```

```
count           ...
```

```
mean            ...
```

```
std             ...
```

```
min             ...
```

```
25%             ...
```

```
50%             ...
```

```
75%             ...
```

```
max             ...
```

The numerical results depend on the actual review dataset.

Explanation

The re module extracts tokens and numeric expressions.

NLTK supplies the English stopword list.

The analyze_review() function calculates five statistics for every review.

pandas.concat() adds these measurements to the dataset.

Seaborn's histplot() displays the distribution of each measurement.

describe() provides descriptive statistics for interpretation.

Interpretation: A concentration of low word counts suggests that many customers leave short reviews. High stopword counts generally accompany longer reviews. Numeric tokens may represent prices, model numbers, ratings, or product specifications. Average word length describes vocabulary structure, not sentiment.

Result

The product reviews were analyzed for stopwords, numeric tokens, word counts, character counts, and average word length. Their distributions and descriptive statistics were generated successfully.

Experiment 4: Sentiment Analysis Using NLTK and VADER

Aim

To calculate and compare the sentiment polarity of product reviews using the NLTK library and the VADER sentiment analyzer, and classify reviews as positive, negative, or neutral.

Theory

Sentiment analysis is a Natural Language Processing (NLP) technique used to identify opinions and emotions expressed in text.

VADER stands for Valence Aware Dictionary and sEntiment Reasoner. It is a rule-based sentiment analysis tool particularly useful for social media text, customer feedback, and informal language.

VADER returns four scores:

Score	Meaning
neg	Proportion of negative sentiment
neu	Proportion of neutral sentiment
pos	Proportion of positive sentiment
compound	Normalized overall sentiment score, ranging from −1 to +1

A compound score close to +1 indicates strongly positive sentiment, while a score close to −1 indicates strongly negative sentiment.

Syntax

Using NLTK's VADER implementation:

from nltk.sentiment.vader import SentimentIntensityAnalyzer

analyzer = SentimentIntensityAnalyzer()

scores = analyzer.polarity_scores("The product is excellent.")

print(scores)

Using the standalone VADER package:

```
from vaderSentiment.vaderSentiment import SentimentIntensityAnalyzer
```

```
analyzer = SentimentIntensityAnalyzer()
```

```
scores = analyzer.polarity_scores("The product is excellent.")
```

```
print(scores)
```

Source code

```
import pandas as pd

import nltk

from nltk.sentiment.vader import SentimentIntensityAnalyzer as
```

```
NLTKVADER

from vaderSentiment.vaderSentiment import (

SentimentIntensityAnalyzer as StandaloneVADER

)

# Download VADER lexicon for NLTK

nltk.download("vader_lexicon", quiet=True)

df = pd.read_csv("product_reviews.csv")

df["review"] = df["review"].fillna("").astype(str)

# Initialize both implementations

nltk_analyzer = NLTKVADER()

vader_analyzer = StandaloneVADER()

def classify(score):

if score >= 0.05:

return "Positive"

elif score <= -0.05:

return "Negative"

return "Neutral"

# NLTK VADER scores

nltk_scores = df["review"].apply(
```

```
nltk_analyzer.polarity_scores

)

df["nltk_positive"] = nltk_scores.apply(lambda x: x["pos"])

df["nltk_negative"] = nltk_scores.apply(lambda x: x["neg"])

df["nltk_neutral"] = nltk_scores.apply(lambda x: x["neu"])

df["nltk_compound"] = nltk_scores.apply(lambda x: x["compound"])

# Standalone VADER scores

vader_scores = df["review"].apply(

vader_analyzer.polarity_scores

)

df["vader_positive"] = vader_scores.apply(lambda x: x["pos"])

df["vader_negative"] = vader_scores.apply(lambda x: x["neg"])

df["vader_neutral"] = vader_scores.apply(lambda x: x["neu"])

df["vader_compound"] = vader_scores.apply(lambda x:
```

```
x["compound"])

df["nltk_sentiment"] = df["nltk_compound"].apply(classify)

df["vader_sentiment"] = df["vader_compound"].apply(classify)

# Display sample results

print(df[[

"review",

"nltk_compound",

"nltk_sentiment",

"vader_compound",

"vader_sentiment"

]].head(10).to_string(index=False))

# Count sentiment categories

print("\nNLTK sentiment distribution:")

print(df["nltk_sentiment"].value_counts())

print("\nStandalone VADER sentiment distribution:")

print(df["vader_sentiment"].value_counts())

# Save output

df.to_csv("sentiment_analysis_results.csv", index=False)

print("\nResults saved to sentiment_analysis_results.csv")
```

Expected output

Example output format:

```
review                          nltk_compound  nltk_sentiment

The product is excellent.          0.5719       Positive

The quality is terrible.          -0.4767       Negative

The product is okay.                0.2263       Positive

NLTK sentiment distribution:

Positive    ...

Negative    ...

Neutral     ...

Results saved to sentiment_analysis_results.csv
```

The scores above are illustrative; actual values depend on the text and VADER lexicon version.

Explanation

The dataset is loaded using Pandas.

NLTK's VADER and the standalone vaderSentiment implementation are initialized.

polarity_scores() calculates the positive, negative, neutral, and compound scores for each review.

The classify() function applies the standard compound-score thresholds.

The resulting sentiment labels are counted and exported to CSV.

Important distinction: Both implementations use VADER. Therefore, they are not independent sentiment models, and their scores will normally match or be very similar. The comparison demonstrates two ways to access VADER, rather than comparing two fundamentally different sentiment-analysis approaches.

Result

Sentiment polarity scores were calculated using NLTK VADER and the standalone VADER package. Product reviews were classified into positive, negative, and neutral categories, and the results were saved for further analysis.

Experiment 5: Scatter Intensity Plot of Sentiments

Aim

To visualize the sentiment intensity of product reviews using a scatter plot, with review index on the X-axis and the VADER compound sentiment score on the Y-axis.

Theory

A sentiment intensity scatter plot displays the distribution and strength of opinions across the collected reviews.

Positive compound scores indicate favorable opinions.

Negative compound scores indicate unfavorable opinions.

Scores close to zero indicate neutral or mixed sentiment.

The standard classification thresholds, +0.05 and −0.05, can be shown as horizontal reference lines. The plot makes it easier to identify strongly positive and strongly negative reviews and observe changes in sentiment across the dataset.

Syntax

```
import matplotlib.pyplot as plt
```

```
plt.scatter(x_values, sentiment_scores)
```

```
plt.axhline(y=0.05, linestyle="--")
```

```
plt.axhline(y=-0.05, linestyle="--")
```

```
plt.show()
```

Source code

The following code uses the results produced in Experiment 4.

```
import pandas as pd

import matplotlib.pyplot as plt

import seaborn as sns

# Load sentiment analysis results

df = pd.read_csv("sentiment_analysis_results.csv")

# Ensure valid compound scores

df["vader_compound"] = pd.to_numeric(

df["vader_compound"], errors="coerce"

)

df = df.dropna(subset=["vader_compound"]).reset_index(drop=True)

# Assign review indices

df["review_index"] = range(1, len(df) + 1)

# Set consistent sentiment colors

palette = {

"Positive": "green",

"Negative": "red",

"Neutral": "gray"

}

plt.figure(figsize=(14, 7))
```

```
sns.scatterplot(
data=df,
x="review_index",
y="vader_compound",
hue="vader_sentiment",
palette=palette,
s=70,
alpha=0.8
)
# Sentiment classification thresholds
plt.axhline(
y=0.05, color="green", linestyle="--",
label="Positive threshold"
)
plt.axhline(
y=-0.05, color="red", linestyle="--",
label="Negative threshold"
)
plt.axhline(
```

```
y=0, color="black", linestyle=":",

label="Neutral reference"

)

plt.ylim(-1.05, 1.05)

plt.xlabel("Review Index")

plt.ylabel("VADER Compound Sentiment Score")

plt.title("Sentiment Intensity of Product Reviews")

plt.legend(loc="best")

plt.grid(alpha=0.25)

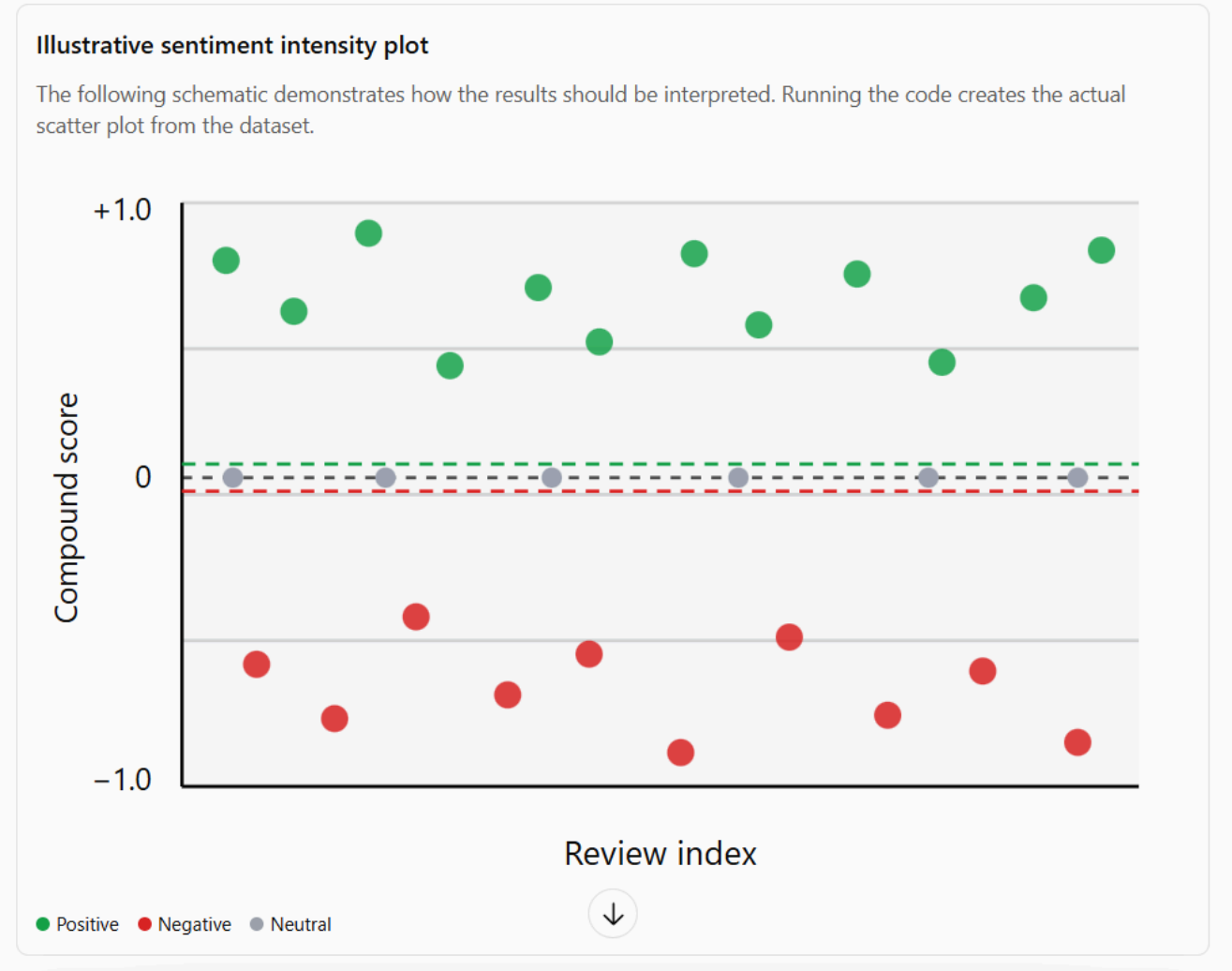
plt.tight_layout()

plt.savefig("sentiment_intensity_scatter.png", dpi=300)

plt.show()

print("Scatter intensity plot saved successfully.")
```

Expected output



Explanation

The X-axis represents each review's position in the dataset.

The Y-axis represents the compound sentiment score, ranging from −1 to +1.

Green points represent positive reviews, red points represent negative reviews, and gray points represent neutral reviews.

The horizontal dashed lines indicate sentiment classification thresholds.

The plot is saved as a high-resolution PNG file at 300 DPI.

The plot is useful for identifying sentiment intensity, but the review index does not represent time unless the data is sorted chronologically.

Result

A scatter intensity plot was generated to visualize the compound sentiment scores of product reviews, making it possible to compare positive, negative, and neutral feedback.

Overall conclusion

The experiments demonstrate the complete workflow for product review analysis:

Web scraping: Extract reviews from an authorized HTML source using Beautiful Soup.

WordCloud analysis: Visualize frequent terms across all reviews and sentiment categories.

Exploratory Data Analysis: Calculate and plot stopword frequency, numeric-token frequency, word count, character count, and average word length.

Sentiment analysis: Calculate compound scores and classify reviews using NLTK VADER and the standalone VADER package.

Sentiment visualization: Plot sentiment intensity to identify positive, negative, and neutral feedback.

The experiments combine web scraping, text preprocessing, statistical analysis, NLP, and data visualization to understand customer opinions and product feedback.

Output files generated

Filename	Purpose
product_reviews.csv	Extracted review dataset
product_reviews_analyzed.csv	Reviews with sentiment labels
review_statistics.csv	Text statistics for each review
review_statistics_distributions.png	Distribution plots
wordcloud_all.png	WordCloud for all reviews
wordcloud_positive.png	WordCloud for positive reviews
wordcloud_negative.png	WordCloud for negative reviews
sentiment_analysis_results.csv	Sentiment scores and labels
sentiment_intensity_scatter.png	Sentiment scatter plot

Practical note: The source HTML must match the CSS selectors in Experiment 1. Run the experiments in sequence so each subsequent program can access the files created by the previous experiment. The output examples are illustrative; actual numerical results and plots will be produced when the programs are executed on a real dataset.

8. Web Scraping using Beautiful Soup

a. Scrape Amazon Reviews

```
import requests
from bs4 import BeautifulSoup

headers = {"User-Agent": "Mozilla/5.0"}
url = "https://www.amazon.in/product-reviews/B09G9BL5CP/"
r = requests.get(url, headers=headers)
soup = BeautifulSoup(r.content, 'html.parser')
reviews = soup.find_all('span', {'data-hook': 'review-body'})
for review in reviews[:5]:
print(review.text.strip())
```